

1.7 Creating applications

You can create two types of WPF applications in Visual Studio: stand-alone applications and browser-based applications. To create a stand-alone WPF application, you use the WPF application template when you create the project.

Visual Studio uses this template to create a fully functional, stand-alone WPF application that contains one window. Next you see the design view of the window and the XAML representation of the same window. The XAML specifies the class that contains the code for the window, the XML schemas that the XAML markup uses, and the properties of the window, such as the title and size.

Visual Studio also adds a Grid control to your window by default. This control acts as a container for the other controls that you place in the window and determines where they appear. You can add controls to a WPF application by dragging the control from the toolbox onto your window or page.

To create a browser-based WPF application, you use the WPF Browser Application template when you create the project. The design environment is almost identical for both application types. The most notable difference between the two applications is that the browser application contains a Page element, whereas the stand-alone application contains a Window element.

You can add controls to browser-based applications in exactly the same way that you add controls to a Windows-based application. The XAML that is generated when you add a control is almost the same in both applications.

1.8 Handling events and commands

You add user interactivity to your applications by using buttons and other controls, and you respond to interactions with these controls in your applications by implementing handlers and commands.

You generate events in WPF in the same way as you would with any .NET Framework technology, such as Windows Forms or Microsoft ASP.NET, by clicking buttons, entering text, changing list selections, setting control focus, or any other similar activity.

In the WPF event model, there are two ways to specify event handlers:

Imperative approach. You can specify event handlers imperatively in code, as you can by using ASP.NET or Windows Forms.

Declarative approach. You can specify the event handler method declaratively